

Supervised Convolutional Neural Network: Using CIFAR-10 to Demonstrate Artificial Learning

Machine Learning

Alexander Schoolcraft, Robbie Ward, Richard Zheng

November 18th, 2025

Model Design

Self-supervised Convolutional Neural Network (CNN) is one of the many networks out there that can be trained without the need of manual labeled data. Traditional machine learning trains neural networks using labeled data, which assigns a tag to each image telling the machine learning (ML) model which object is associated with that image. Self-supervised CNN is a good training method that guarantees the model is only learning the desired features within a dataset while ignoring the irrelevant data points that are not helpful to this specific model. However, presorting data points and compiling a dataset takes an enormous amount of time and effort, which makes the traditional training method inefficient. This is where self-supervised convolution neural networks come in handy to take out the hassle of presorting thousands of data points.

The first feature extraction phase focus on the low-level spatial features from raw images through convolutional layers. There are 60000 CIFAR-10 images being used to train the model. These images are each represented as a 32x32x3 tensor, where each of these values represents the RGB intensities. The first convolutional layer was used to allow the model to learn basic visual patterns like its edges, corners, and color gradients. Therefore, 32 filters with a kernel size of 3x3x3 were used to extract these features. During this training phase, the model computes feature maps, captures local patterns that do not change with positions from the filters sliding across the images spatially. Before the image is passed onto the second convolutional layer, a Rectified Linear Unit (ReLU) activation function is used to bring in non-linearity. This allows the model to learn more complex visual representations that cannot be captured using linear transformations.

Once the basic visual patterns were extracted from the first convolutional layer, these images were filtered through the second convolutional layer. This layer has the same number of filters as the first layer, but with a $3 \times 3 \times 32$ kernel size this time. The increment in kernel size helps this second layer to learn more abstract and complex visual patterns, such as curves, textures, and partial object shapes like fur texture, animal limbs, wings, and wheels. By learning these finer features about the images, it helps the model to begin distinguishing meaningful visual components based on categories rather than simple pixel level patterns. To introduce non-linearity into this layer, a ReLU activation function is used again to allow the model to learn hierarchical feature representations similar to the first layer. By the time the training dataset has passed through this layer, the model has extracted a set of useful feature maps ready to be used for spatial dimensionality reduction.

Spatial dimensionality reduction is a key step in the learning process that reduces the spatial dimensions of the feature map. This process is done using a max pooling layer with a size of 2×2 and a stride of 2, effectively narrowing the input. By decreasing the spatial resolution, this process has helped decrease the number of parameters and computational complexity significantly, increasing the model's efficiency while maintaining the same level of performance. Another benefit that max pooling layer brings to this model is its ability to provide spatial invariance. Since the CIFAR-10 datasets are compiled with various object images in various positions, orientation, and scales, the inclusion of this pooling layer trains the model to be less sensitive to minor distortions in input images. Similar to the ANN, this model also included a dropout layer after pooling. The dropout layer in this model deactivates 30% of randomly selected neuron during each forward pass. This dropout layer serves the same purpose as any

other dropout layer in other neural networks, which forces the model to learn more robust and generalized feature representations from the dataset rather than memorizing individual samples.

Since the spatial feature extraction produced a two-dimensional feature map, it needs to be flattened into a one-dimensional vector with 4096 units so it can be processed by two fully connected layers. The first dense layer contains 256 neurons, and they are responsible for learning abstract representation of the dataset by incorporating and weighting features that were extracted earlier in a meaningful way. Similar to the first learning stage, the ReLU activation function was used to introduce non-linearity from the training resulted from this layer. To further avoid this neural network from memorizing the exact training sample and to prevent overfitting, a dropout layer is applied to deactivate 20%-30% of neurons randomly during training. This is to make sure that the neural network is relying on the features that was acquired through training and not the specific training samples. Following the dropout layer, the output is once again being passed through a fully connected layer with 10 neurons. These 10 neurons were equally distributed to be used by each of the 10 classes in the CIFAR-10 dataset. SoftMax activation function was also incorporated into the model to convert outputs into probability distributions, assigning a simple confidence score to each of the ten categories. This allows the model to output a confident class with the highest probability.

Hyperparameter Tuning

The two main types of hyperparameters in this model are the numerical and categorical hyperparameters. These two hyperparameters are crucial in controlling how this model learns, regularizes, and generalizes data from the training dataset. Therefore, tuning these two hyperparameters before training is crucial to ensure convergence and final accuracy.

The numerical hyperparameter includes the model's learning rate, dropout rate, weight decay, number of training epochs, batch size, and patience. All of these variables directly correlate to the speed, regularization strength, and stability of the training process. A learning rate that is too high can cause the model not able to acquire the full pattern from the dataset, while a learning rate that is too low can unnecessarily prolong the training epochs and affect model convergence. While batch size and patience parameters do not impact the model's learning process as much as manipulating the model's learning rate, they provide the model with stability and efficiency during training, with the patience parameter stopping the model training when the validation accuracy stops improving.

Categorical hyperparameters define the training approach of the model. This includes choosing the best optimizer (e.g. AdamW or SGD), learning rate scheduler (e.g. Cosine, OneCycle, or None), and the strength of data augmentation (Strong, Basic, or None). These hyperparameters directly impact on the model and how it updates its weights, learning rate variations, and generalization over the entire dataset. Additionally, these categorical hyperparameters allow the model to learn from an expanded dataset by creating new images through random crops, flips, noise injections, rotations, and contrast jitters on the existing images. Through exposing the model to learn about these augmented images, it enhances the model's stability by enhancing its resilience to changes in lighting, positions, and backgrounds.

A smaller scale auto-tuning process was incorporated on top of the manually selected hyperparameters to systematically search for the optimal combinations. This process is done through evaluating different configurations and selecting the ones with the highest validation accuracy. Based on the tuner, AdamW optimizer, OneCycle learning rate scheduler and strong

data augmentation presents the most stable convergence and highest generalization performance when training the model with the CIFAR-10 dataset.

Training and Evaluation

The CIFAR-10 dataset was divided into three sets - one for training, one for testing, and the other for validating the model. The training set is used to backpropagate the model to optimize the model weights, while the validation set was to prevent model overfitting during the training process by monitoring its generalization performance. Finally, the testing set is exclusively used during the testing process to ensure the neural network's accuracy is free from the influence of prior exposures.

The images used to train this model were preprocessed before training starts. This process includes normalizing and resizing the images and converting raw pixel values into suitable input formats for the neural network. A strong data augmentation was also performed in addition to these standard preprocessing stages to enhance the network's resilience on irregularities during testing process. This includes modifying the pre-existing images in the dataset by randomly flipping, cropping and erasing parts of the images, jittering the brightness and contrast, and rotating images to create new training samples.

From the auto-tuning process, the AdamW optimizer with the OneCycle Learning rate scheduler was selected to optimize the output of the model. This combination helped maintain a stable training process by adjusting its learning rate dynamically to accelerate convergence in the beginning of training and fine-tuning weights in the later training phases. To improve efficiency and maintain a stable gradient update, this model was trained for a fixed number of epochs. Both the accuracy and loss metrics were assessed during each training epoch for the training and

validation sets. These two metrics track the model's learning behavior, its convergence stability, and potential signs of overfitting. To evaluate the accuracy of the model after training has been completed, class-level metrics like precision, recall, and F1-score were used to evaluate the testing dataset. To visualize the model's accuracy, image samples were tested on this model. This test allows the programmers to see the model's strength and its limitations when it comes to pixelated image recognition.

Results

This convolutional neural network trained on the CIFAR-10 dataset was evaluated both quantitatively and qualitatively through multiple metrics. The quantitative evaluation includes measuring the training and validation loss, precision, recall, and F1-Score. The qualitative performance of this model was also evaluated by feeding pixelated images into the model and letting the model predict what the content is.

By analyzing the training and validation accuracy recorded over the 90 training epochs, the model has demonstrated a steady learning progression. When training begins, the model's training and validation accuracies grew exponentially as the model acquired basic visual features

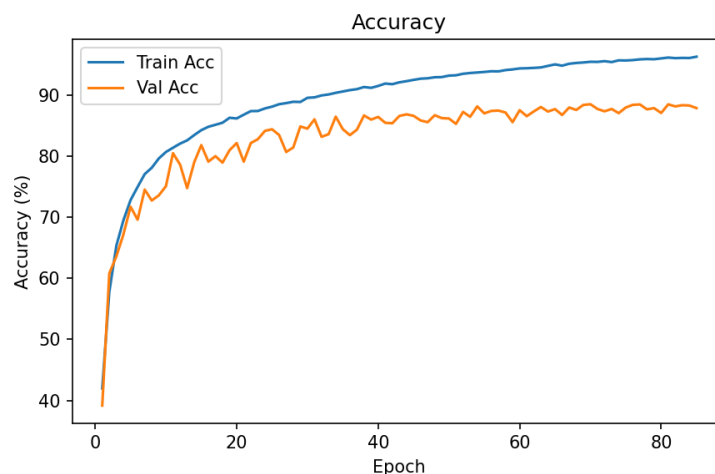


Figure 1: Training and Validation Accuracy

of the image dataset like edges, textures, and color transitions. As the training continues and the model keeps learning these features, these accuracy curves begin to slowly level off. Even with a small difference between the training and validation accuracy curve at the end of training, this model has performed well in generalizing unknown data with minimal overfitting.

Similar to any other neural networks, they all experience a sharp decrease in loss value at the beginning of training. This is normal in all neural networks as they are learning about the fundamental visual features of the images. However, when training progresses, the model slowly transitions from basic feature extraction to fine-grain tuning using the optimizers. This has caused the training loss to drop exponentially during the initial training stage and flattens as optimization starts. For our model, pre-configured learning rate scheduling and optimization strategies have performed exceptionally well in handling data loss during training with our model's efficient feature learning and stable weight updates.

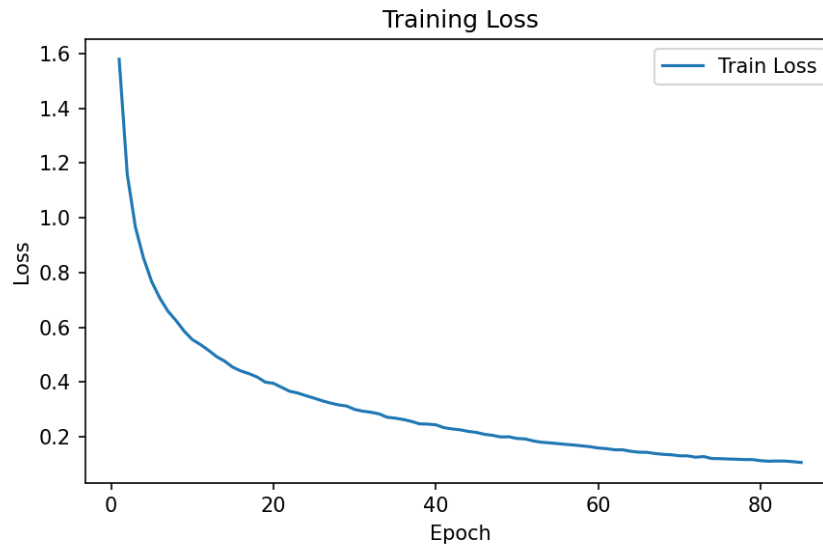


Figure 2: Training loss curve

The last quantitative performance was assessed through evaluating the recall, precision and F1-score values across all ten classes in the CIFAR-10 dataset. This model has performed

well statistically in classes with distinctive visual features such as airplanes, automobiles, ships, and trucks with a high F1-score. However, predictions made in categories like cat and dog have a lower F1-score, indicating that similarities between classes and background clutter have presented challenges for accurate prediction. Despite the model's low F1-score when predicting images within a specific category, the high macro-average and weighted-average F1-scores still confirms the model has demonstrated a strong overall classification capability with consistent generalization.

		precision	recall	f1-score	support
1					
2	airplane	0.8989205103042198	0.916	0.9073798910351659	1000.0
3	automobile	0.9353905496624879	0.97	0.9523809523809523	1000.0
4	bird	0.8142470694319206	0.903	0.8563300142247511	1000.0
5	cat	0.731433506044905	0.847	0.7849860982391103	1000.0
6	deer	0.8804347826086957	0.891	0.885685884691849	1000.0
7	dog	0.8915254237288136	0.789	0.8371352785145889	1000.0
8	frog	0.8999028182701652	0.926	0.9127649088220798	1000.0
9	horse	0.9688888888888889	0.872	0.9178947368421052	1000.0
10	ship	0.9558067831449126	0.93	0.94272681196148	1000.0
11	truck	0.9863325740318907	0.866	0.9222577209797657	1000.0
12	accuracy	0.891	0.891	0.891	0.891
13	macro avg	0.8962882906116899	0.891	0.8919542297691848	10000.0
14	weighted avg	0.89628829061169	0.891	0.8919542297691849	10000.0

Figure 3: Precision, recall, and F1-score metric

For qualitative evaluation, the model was tested on a set of correctly classified images to evaluate the model's accuracies when recognizing real visual patterns. From the result, the model has proved its ability to identify general features of the images, including its shape, color distribution, texture, and contextual cues. The model has shown an exceptional high confidence when predicting images in the visually distinct categories, including cars, trucks, airplanes, ships, and frogs. This is a sign that the model has been successfully trained with strong feature extraction and generalization capabilities.

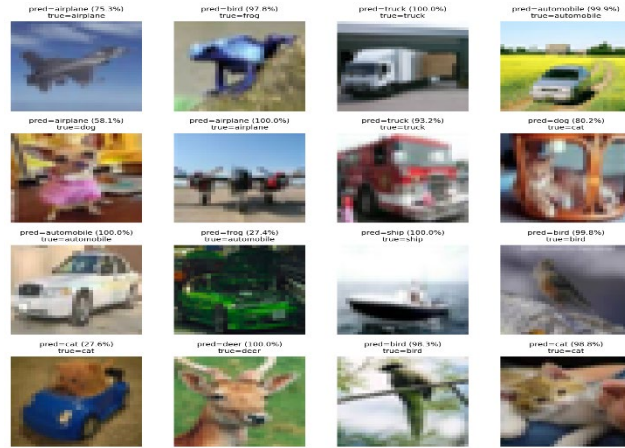


Figure 4: Random selection of CIFAR-10 images and their classifications

To further investigate the model's limitations, a set of images that were classified incorrectly were tested on the model to analyze at which point during training the model encountered limitations. From the result, there are a handful of cases where the model is highly confident on a misclassification. This means that some of these errors are linked to objects sharing similar color tones, shapes, or textures of the other classes. There are also cases where the misclassification was due to the background elements being closely related to features of other classes. For example, in the CIFAR-10 dataset, frogs and deer as well as cats and dogs were confused with each other respectively due to the fact that they either share a similar color

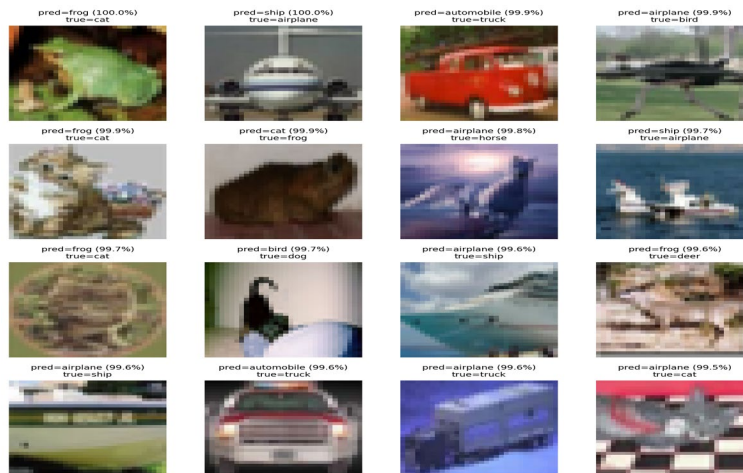


Figure 5: List of incorrectly labeled images with confidence score

pattern or fur structures. These indicate that the model was trained well on basic feature extraction, but it still needs more precise tuning to discover the finer-grained similarities.

Discussion & Future Work

Based on the analysis of the qualitative and quantitative performance of this model, it is evident that this model has effectively acquired algorithms on how to represent visual aspects from the CIFAR-10 dataset in a meaningful way with a strong classification performance. This can be seen from multiple performance vectors of the model including its training, validation, and loss curves. The trends from these curves not only suggest that the model is effectively learning, but it is also generalizing and converging learned data throughout the entire training process. The downward trending loss curve further confirms that the model is learning efficiently, with the model trying and acquiring the majority of the data instead of dropping them during training.

The trained model has achieved a high performance when distinguishing images from visually distinct classes like automobiles, planes, trucks and ships, obtaining an F1-score over 0.90. The feature extraction process has been simplified by clear structures, consistent color distributions, and recognizable object boundaries from these categories. However, when it comes to recognizing images within the cat and dog categories, the model performance was weaker due to the fine-grained similarities in textures, shapes, and background color.

Future work on this model can include incorporating deeper architectures like Residual Network to help with the fine-grained feature extraction. ResNet uses residual connection to enable deeper networks to retain and refine both low and high-level visual features, allowing the

model to capture even the tiniest class-specific differences that were not picked up by AdamW optimizer with the OneCycle Learning rate scheduler.